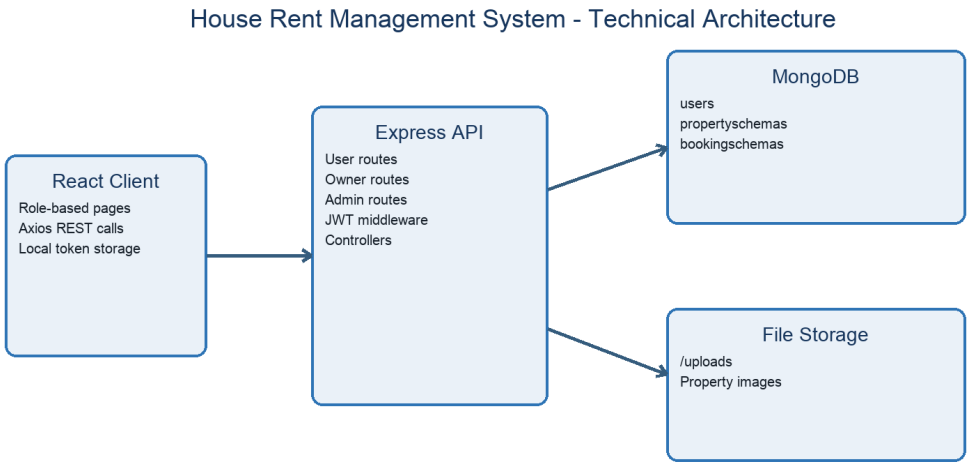
FULL STACK DEVELOPMENT WITH MERN

House Rent Management System

HouseHunt / RentEase



Project Documentation

Submitted by

Duvva Naveen (Team Lead)

Addepalli Ramya Hari Priya Varshini (Team Member)

Nandikolla Usha Sri (Team Member)

Haripriya Godaba (Team Member)

Narayana Murthy Perapu (Team Member)

Under the guidance of M. Sarit Sir | July 2026

1. Introduction

Field	Details
Project Title	House Rent Management System (HouseHunt / RentEase)
Repository	https://github.com/naveenduvva692004-collab/house-rent-master
Academic Context	Full Stack Development with MERN
Team Lead	Duvva Naveen
Team Members	Addepalli Ramya Hari Priya Varshini; Nandikolla Usha Sri; Haripriya Godaba; Narayana Murthy Perapu

This report is based on the repository's current code, the supplied project document, the required Google Drive MERN documentation format, and the phase structure demonstrated in the reference project. Inaccurate copied references from the earlier draft have been removed.

2. Project Overview

Purpose

The House Rent Management System centralizes property discovery, owner listing management, rental booking requests, and administrator oversight in a single MERN web application. It is intended as an academic full-stack demonstration rather than a production real-estate marketplace.

User Roles

Role	Capabilities
Renter	Register, log in, browse/filter properties, submit booking details, and view booking history.
Owner	Register, await administrator approval, manage property listings/images, review bookings, and update status.
Admin	View users, approve or revoke owner access, and review all properties and bookings.

Implemented Features

- Role-aware registration and navigation.
- bcrypt password hashing and JWT authentication.
- Property browsing and filters for address, property type, and ad type.
- Owner property CRUD and multiple image upload.
- Booking request and booking-history flow.
- Administrator approval and monitoring views.

3. Architecture

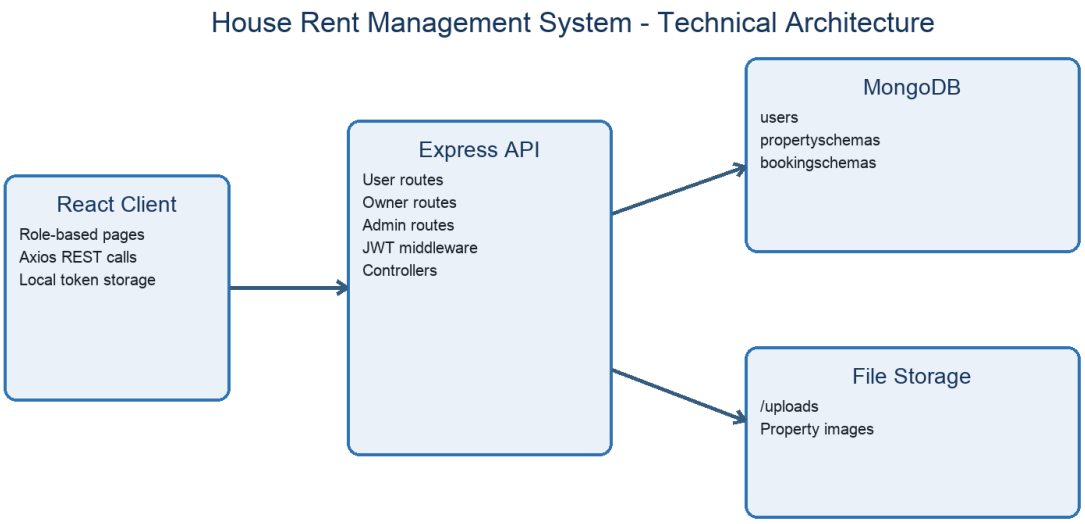


Figure 1. Technical architecture derived from the repository.

Frontend

React 18 renders public pages and role dashboards. React Router defines routes, Axios calls the API, and Material UI, Ant Design, Bootstrap, and React Bootstrap provide interface components. Login information and the JWT are stored in localStorage.

Backend

Node.js and Express expose user, owner, and administrator routes. Route modules call controllers, and a JWT middleware protects selected operations. Express also serves property images from the backend/uploads directory.

Database

Mongoose connects the API to MongoDB. Three collections model users, properties, and bookings. The current schemas are flexible because strict mode is disabled, which accommodates fields such as granted and isAvailable but also permits spelling inconsistencies.

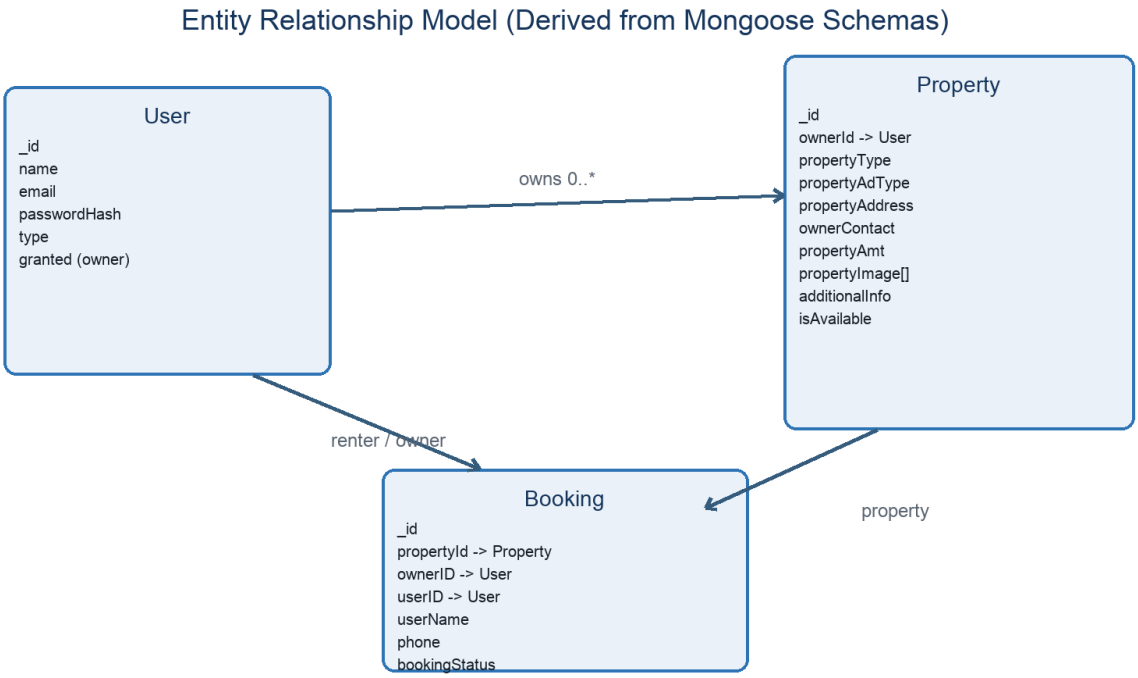


Figure 2. Entity relationship model reconstructed from Mongoose schemas.

4. Setup Instructions

Prerequisites

- Node.js with npm.
- MongoDB Community Server or a MongoDB Atlas database.
- Git.
- Modern browser and code editor.

Installation

```
git clone https://github.com/naveenduvva692004-collab/house-rent-master.git
cd house-rent-master/backend
npm install
# create backend/.env (see below)
npm start
# new terminal
cd house-rent-master/frontend
npm install
npm start
```

Backend .env

```
PORT=8001
MONGO_DB=mongodb://127.0.0.1:27017/house_rent
JWT_KEY=replace_with_a_long_random_secret
```

5. Folder Structure

```
frontend/src/
├── App.js                        # routes and role dashboards
├── images/                       # bundled images
└── modules/
    ├── common/                   # home, register, login, forgot password
    ├── admin/                    # users, properties, bookings
    └── user/{Owner,renter}/      # role-specific pages

backend/
├── config/connect.js             # MongoDB connection
├── controllers/                  # business logic by role
├── middlewares/authMiddlware.js # JWT validation
├── routes/                       # Express route definitions
├── schemas/                      # Mongoose data models
├── uploads/                      # property images
└── index.js                      # application entry point
```

6. Running the Application

Service	Directory	Command	Default Address
Backend API	backend	npm start	http://localhost:8001 when PORT=8001
Frontend	frontend	npm start	http://localhost:3000

The frontend source currently hard-codes port 8001. If PORT is omitted, the backend falls back to 8000 and client requests will fail. Use PORT=8001 or refactor the client to read REACT_APP_API_BASE_URL.

7. API Documentation

Method	Endpoint	Auth	Purpose
POST	/api/user/register	Public	Create renter or owner account
POST	/api/user/login	Public	Authenticate and return JWT + user
POST	/api/user/forgotpassword	Public	Replace password by email (unsafe as implemented)
GET	/api/user/getAllProperties	Public	List all properties
POST	/api/user/getuserdata	JWT	Return authenticated user
POST	/api/user/bookinghandle/:propertyid	JWT	Create pending booking
GET	/api/user/getallbookings	JWT	List bookings for authenticated renter

Method	Endpoint	Auth	Purpose
POST	/api/owner/postproperty	JWT + multipart	Create property with images
GET	/api/owner/getallproperties	JWT	List authenticated owner's properties
GET	/api/owner/getallbookings	JWT	List bookings for authenticated owner
POST	/api/owner/handlebookingstatus	JWT	Update booking and availability
DELETE	/api/owner/deleteproperty/:propertyid	JWT	Delete property
PATCH	/api/owner/updateproperty/:propertyid	JWT + multipart	Update property
GET	/api/admin/getallusers	JWT	List all users
POST	/api/admin/handlestatus	JWT	Grant or revoke owner access
GET	/api/admin/getallproperties	JWT	List all properties
GET	/api/admin/getallbookings	JWT	List all bookings

Example Login Request

```
POST /api/user/login
{
  "email": "renter@example.com",
  "password": "example-password"
}
```

Example Success Shape

```
{
  "message": "Login success successfully",
  "success": true,
  "token": "<jwt>",
  "user": { "_id": "...", "name": "...", "email": "...", "type": "Renter" }
}
```

8. Authentication and Authorization

1. Registration hashes the password with bcrypt using a generated salt.
2. Login compares the supplied password to the stored hash.
3. A JWT containing the user ID is signed with JWT_KEY and expires after one day.
4. Protected requests send Authorization: Bearer <token>.
5. The middleware verifies the token and writes the decoded ID to req.body.userId.

Security limitation: the backend verifies identity but does not enforce role-based authorization. An authenticated renter could attempt owner or admin endpoints. Add server-side role middleware before production use.

9. User Interface

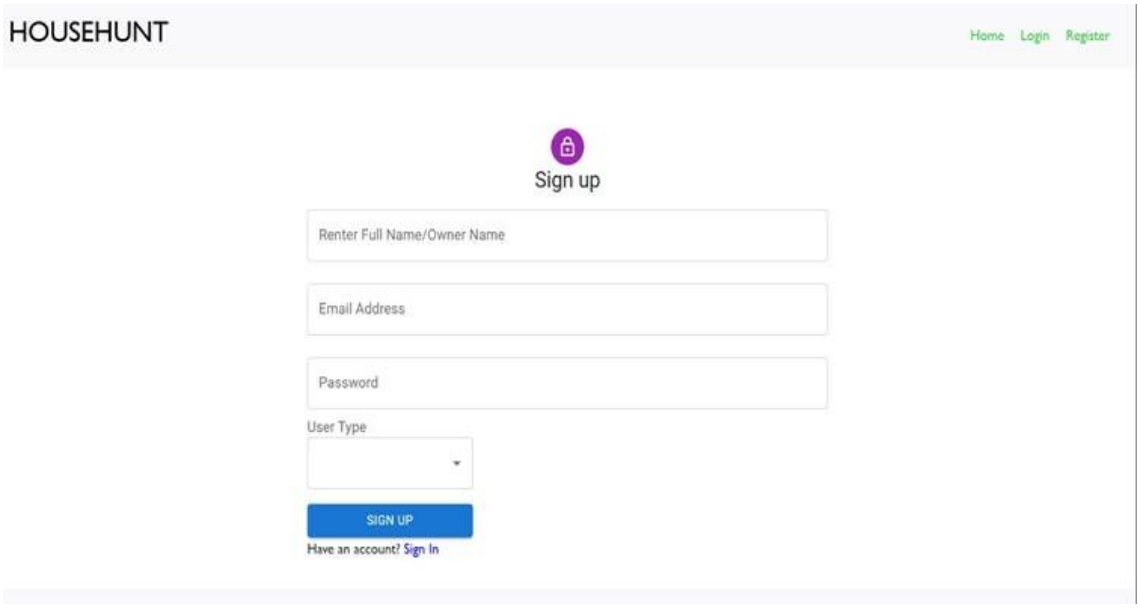


Figure 3. Registration interface from the supplied project document.

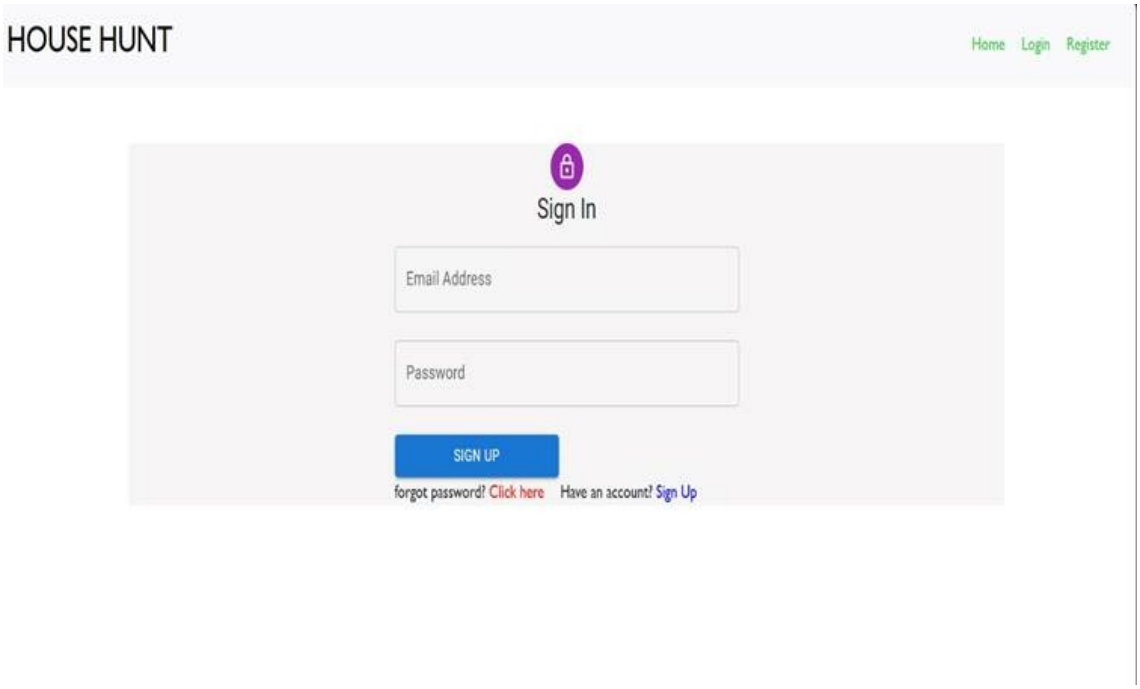


Figure 4. Login interface from the supplied project document.

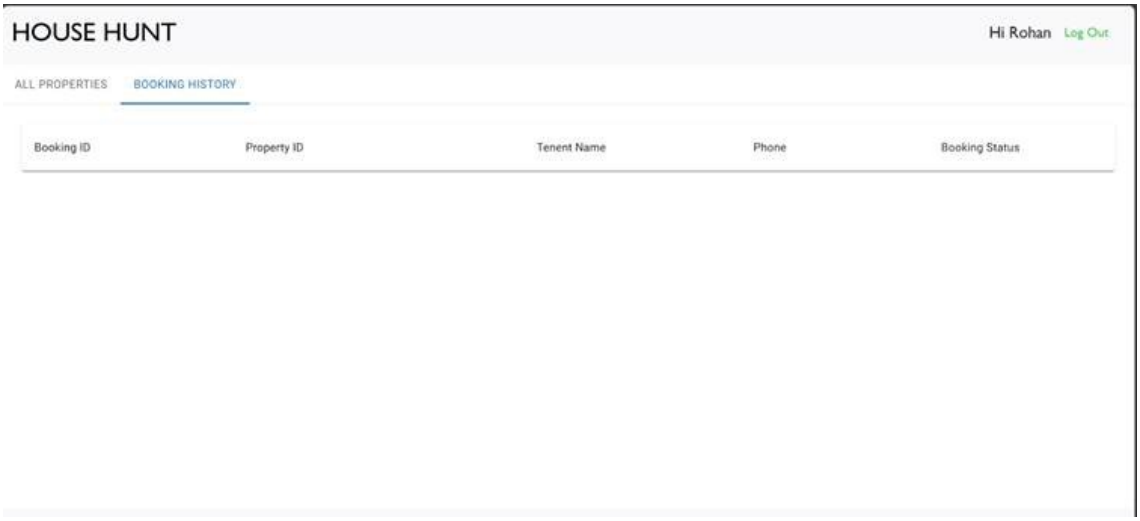


Figure 5. Renter booking-history interface from the supplied project document.

The public application also displays property cards with image, address, type, ad type, amount, contact, availability, and additional information. Owners and administrators use data tables for management tasks.

10. Testing

No reproducible automated test suite or performance report exists in the repository snapshot. The backend package.json test command explicitly exits with 'no test specified'. Therefore, this documentation does not claim fabricated pass rates or response times.

Test Area	Recommended Tool	Minimum Acceptance
API functional	Postman/Newman	All endpoints return expected status and response schema
Authorization	Jest/Supertest	Wrong roles receive 403; missing/invalid JWT receives 401
UI	React Testing Library	Registration, login, filtering, and booking flows render and handle errors
Load	k6/JMeter	At 25 concurrent read users, p95 < 2 seconds and error rate < 1%
Security	OWASP ZAP + review	No unauthenticated state change; verified reset; restricted uploads

11. Screenshots and Demo

Source repository: https://github.com/naveenduvva692004-collab/house-rent-master

The earlier draft contains a Google Drive demo link. Confirm that the team owns the video and that sharing permissions work before including it in the final submission.

12. Known Issues

Issue	Evidence / Impact	Recommended Fix
API port mismatch	Frontend uses 8001; backend fallback is 8000	Environment-based API base URL and matching PORT
Missing role authorization	JWT middleware checks only identity	Require Admin/Owner/Renter roles per route
Unsafe password reset	Anyone knowing an email can set a new password	Use time-limited verified reset tokens
Booking field mismatch	Schema declares propertId while controllers/UI use propertyId	Rename consistently and migrate data
Flexible schemas	strict:false allows accidental fields	Define explicit granted/isAvailable and use strict schemas
Local uploads	Original filenames can collide; not horizontally scalable	Unique names, type/size limits, cloud object storage
In-memory filtering	Controllers fetch all records and filter in JavaScript	Query MongoDB by owner/user ID and add indexes
No tests	No backend tests and no project UI tests found	Add unit, integration, UI, and load tests

13. Future Enhancements

- Server-enforced role authorization and audit logging.
- Verified email/password recovery and stronger validation.
- Saved searches, favorites, alerts, and owner/renter notifications.
- Geospatial search, maps, and neighborhood information.
- Payment, agreement, identity-verification, and dispute workflows.
- Cloud file storage, deployment configuration, CI/CD, logging, and monitoring.
- Automated test coverage and measurable performance baselines.
- Responsive accessibility improvements and clearer error feedback.

Conclusion

The repository demonstrates the core pieces of a MERN rental workflow: a React client, an Express API, MongoDB persistence, JWT authentication, property management, and booking operations. The system is appropriate for an academic demonstration, while the known security, data-quality, testing, and deployment gaps should be addressed before any real-world use.

Submission note: Team ID and any member-specific role allocation should be confirmed before final academic submission.